

Relatório Técnico: Implementação de CRUD via RMI com Pyro5

Aluno: João Pedro Cunha Guska
RA: 2264994

7 de maio de 2026

1 Introdução

Sistema **CRUD Governamental de Transporte** para o uso de *Middleware* de **RMI (Remote Method Invocation)**, utilizando o **Pyro5**. O sistema permite o gerenciamento de rotas e modais de transporte público (trens, metrô, ônibus, VLTs e balsas), automatizando a persistência e a recuperação de dados de infraestrutura.

2 Estrutura do Banco de Dados

A persistência é realizada através do **SQLite3**. A tabela `linha_transporte` possui a seguinte estrutura de campos:

- **id**: Chave primária com autoincremento.
- **rota**: Identificador textual da rota.
- **modal**: Tipo de veículo utilizado.
- **capacidade**: Lotação máxima (inteiro).
- **extensao**: Comprimento da rota em quilômetros (real).
- **tarifa**: Valor da passagem (real).

3 Arquitetura e Protocolo RMI

O uso do Pyro5 permite a invocação de métodos remotos de forma transparente. A arquitetura foi estruturada em classes, onde o servidor disponibiliza um objeto remoto contendo a lógica de negócio do CRUD.

A transferência de dados entre cliente e servidor é realizada através da serialização de **Dicionários Python**, garantindo que a estrutura de dados seja preservada sem a necessidade de definir protocolos de bytes fixos. O protocolo de serialização utilizado é o **serpent**, que converte os objetos em um formato legível e seguro para tráfego via TCP.

4 Análise de Tráfego via Wireshark

A análise do comportamento do sistema foi realizada através da captura de pacotes na interface de *loopback* utilizando a ferramenta *tcpdump* no ambiente Linux (WSL), com posterior inspeção no Wireshark.

4.1 Handshake com o NameServer

A comunicação inicial ocorre com o serviço de nomes do Pyro (*NameServer*), operando por padrão na porta 9090.

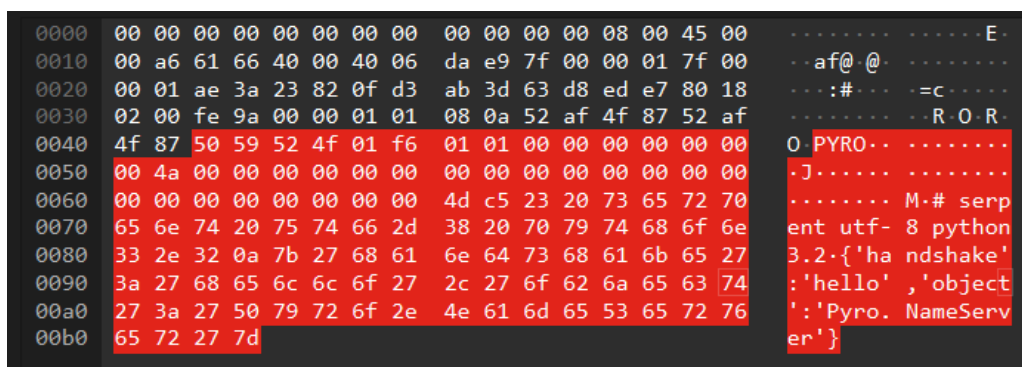


Figura 1: Estabelecimento de conexão TCP e Handshake inicial.

Observa-se na Figura 1 o *Three-way handshake* do protocolo TCP (SYN, SYN-ACK, ACK). Após a conexão, verifica-se o envio do pacote de handshake 'hello' do Pyro. O servidor responde com os dados da interface, permitindo que o cliente identifique os métodos e atributos disponíveis para interação remota.

4.2 Resolução de Nomes e Localização do Objeto

Para que o cliente possa realizar chamadas ao servidor de transporte, é necessário localizar a URI (*Uniform Resource Identifier*) do objeto registrado.

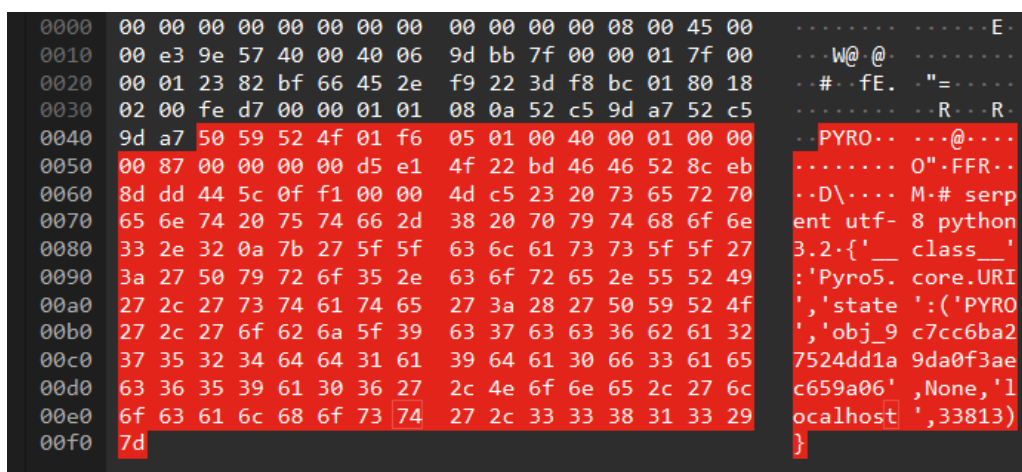


Figura 2: Processo de Lookup e retorno da URI do Servidor.

A Figura 2 demonstra a invocação do método `lookup` pelo cliente, solicitando o endereço do objeto associado ao nome `joaoguska.CRUD`. A resposta do *NameServer* contém a URI completa, indicando que o objeto remoto está disponível no `localhost` em uma porta dinâmica (ex: 33813).

4.3 Invocação do Método de Inserção (Create)

Com a localização do servidor estabelecida, o cliente realiza a chamada do método `inserir`.

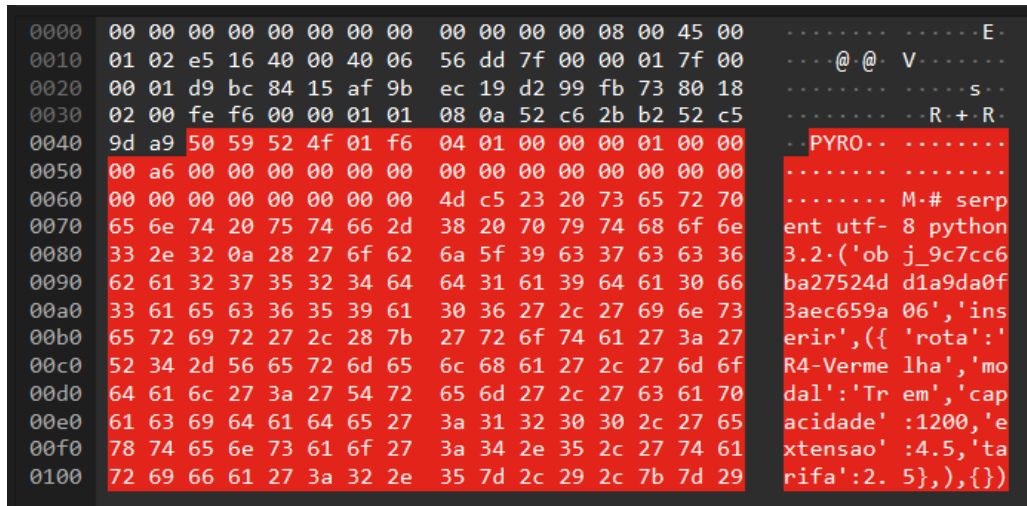


Figura 3: Carga útil do pacote contendo a chamada do método e dados serializados.

Na Figura 3, é possível analisar o *payload* do pacote TCP. Se nota a presença do identificador do método (`'inserir'`) e o conteúdo dos parâmetros enviados. O dicionário contendo os dados da rota (ex: `R4-Vermelha, Trem, Capacidade: 1200`) é transmitido de forma serializada em formato `serpent`, demonstrando a abstração da camada de transporte provida pelo middleware.